Movie Rental Database System

CS 157A — Introduction to Database Management Systems

Team Members:
Addison Schleigh — SID: 017595110
Haitham Assaf — SID: 017558515
Tyler Fabela — SID: 016347201

1. Introduction

This project designs and implements a Movie Rental Database System for a video rental store. The application allows store staff to manage a catalog of movies, register and update customer accounts, process rentals and returns, and record customer ratings. The system follows a three-tier architecture: a presentation layer built with Java Swing, an application layer containing JDBC-based DAO classes, and a data layer consisting of a MySQL relational database whose schema is normalized to Boyce-Codd Normal Form (BCNF).

Managing a video rental store involves coordinating data across several domains: which titles are in the catalog, who the customers are, which copies are currently checked out, when copies are returned, and how customers rate what they have watched. A properly normalized relational database eliminates redundancy, enforces data integrity through primary and foreign key constraints, and supports efficient queries through structured SQL access. Building this system as a three-tier application also illustrates the principle of *data independence*: changes to the user interface, business logic, or storage representation can each be made without disturbing the others.

2. Objectives

Primary goals.

• Design a relational schema with at least four tables, all in BCNF, with proper primary key, foreign key, and NOT NULL constraints.
• Populate each table with at least fifteen rows of representative sample data.
• Build a Java application that connects to MySQL through JDBC and executes SELECT, INSERT, UPDATE, and DELETE operations from each table.
• Provide a graphical user interface that lets a non-programmer perform every database operation through forms and buttons.

Functionality overview. The application supports browsing and searching the catalog, registering customers, processing rentals and returns, and recording ratings. Staff can also filter active rentals, look up ratings for a specific movie, and compute the average score a movie has received.

3. Database Design

3.1 Entity-Relationship Model

The conceptual model identifies two strong entity sets, two associative entity sets, and one weak entity set, connected by five relationships. **Customer** and **Movie** are the strong entities. Each customer can rent many movies, and each movie can be rented by many customers, but a single rental transaction is itself an object with its own identifier and additional attributes (rental date, due date, status). To capture this, we promote the many-to-many relationship between Customer and Movie into an associative entity called **Rentals**,

connected to its participants through the *Makes* and *For* relationships. The same approach is used for **Ratings**, with the *Gives* and *Of* relationships.

Returns is a weak entity. Each return record exists only in association with the rental it belongs to and would not be uniquely identifiable on its own. The identifying relationship *Has* connects Returns to Rentals.

Table 1 summarizes the cardinalities.

Relationship	Connects	Cardinality
Makes	Customer → Rentals	1 : M
For	Rentals → Movie	N : 1
Has	Rentals → Returns	1 : 1
Gives	Customer → Ratings	1 : M
Of	Ratings → Movie	N : 1

3.2 Relational Schema

Each entity set in the E/R diagram becomes a table; relationships contribute their attributes and the keys of their connected entity sets as foreign keys. This produces five relations:

Relation	Attributes (PK underlined, FK in italics)
Movies	MovieID, Title, Genre, Director, ReleaseYear, MPAARating, CopiesAvailable, RentalPrice
Customers	CustomerID, FirstName, LastName, Email (UNIQUE), Phone, Address, JoinDate
Rentals	RentalID, *CustomerID*, *MovieID*, RentalDate, DueDate, Status
Returns	ReturnID, *RentalID* (UNIQUE), ReturnDate, LateFee
Ratings	RatingID, *CustomerID*, *MovieID*, Score, ReviewDate

3.3 BCNF Justification

A relation R is in BCNF if for every nontrivial functional dependency X → Y that holds in R, X is a superkey of R. We verify each relation:

• **Movies.** The only nontrivial FDs are MovieID → (all other attributes). MovieID is the primary key, hence a superkey. BCNF holds.

• **Customers.** Both CustomerID and Email are unique, so CustomerID → (rest) and Email → (rest). Both are superkeys. BCNF holds.

• **Rentals.** The only nontrivial FD is RentalID → (rest). RentalID is the primary key. BCNF holds.

• **Returns.** ReturnID → (rest), and RentalID is declared UNIQUE so RentalID → (rest) as well. Both are superkeys. BCNF holds.

• **Ratings.** RatingID → (rest), and RatingID is the primary key. BCNF holds.

Because every nontrivial dependency is anchored in a superkey, none of the schemas exhibit the redundancy or update anomalies that BCNF is designed to prevent.

4. Implementation

4.1 Three-Tier Architecture

The application is divided into three independent layers, each in its own Java package:

• **Presentation layer (package `ui`).** A Swing-based GUI built around a JFrame containing a JTabbedPane. Each tab corresponds to one entity (Movies, Customers, Rentals, Returns, Ratings) and exposes a JTable view plus buttons for Add, Edit, Delete, and Search.

• **Application layer (packages `dao` and `model`).** Plain Java model classes mirror table rows; DAO (Data Access Object) classes encapsulate the SQL for each table. The UI never builds a SQL string directly — it asks a DAO for data and hands the DAO an updated model object.

• **Data layer (package `db` and the MySQL server).** The DatabaseConnection class is the single point that obtains a JDBC Connection. The schema and sample data are stored in the MySQL `movie_rental` database.

This separation realizes data independence at the application level: replacing the GUI with a JSP/Servlet front end, or porting the schema to a different RDBMS, only requires changes in one layer.

4.2 JDBC Layer

All database access goes through the standard JDBC API. A connection is obtained from `DriverManager.getConnection` using the MySQL connector. Every DAO method follows the same try-with-resources pattern so that connections, statements, and result sets are released in reverse order even when an exception is thrown:

```
try (Connection conn = DatabaseConnection.getConnection();
     PreparedStatement pstmt = conn.prepareStatement(sql)) {
     pstmt.setInt(1, movieId);
     try (ResultSet rs = pstmt.executeQuery()) {
         while (rs.next()) {
             movies.add(buildMovie(rs));
         }
     }
}
```

Every query that takes user input uses a PreparedStatement with parameterized placeholders (?) instead of string concatenation. This avoids SQL injection and lets the database reuse the compiled query plan across calls.

4.3 Required SQL Operations

Each DAO class exposes methods for the four core SQL operations. Examples drawn from the Movies table illustrate the pattern; the other DAOs follow the same template.

Operation	Method	SQL
SELECT	getAllMovies()	SELECT * FROM Movies ORDER BY MovieID
SELECT	searchByTitle(kw)	SELECT * FROM Movies WHERE Title LIKE ?
INSERT	insertMovie(m)	INSERT INTO Movies (...) VALUES (?, ?, ...)
UPDATE	updateMovie(m)	UPDATE Movies SET ... WHERE MovieID = ?
DELETE	deleteMovie(id)	DELETE FROM Movies WHERE MovieID = ?

5. Functional Requirements

Users. Store staff (clerks and managers) are the primary users. They access the system by launching the desktop GUI on a workstation in the store. The application is intended for internal use, not for end customers.

Features and processes.

• **Browse / Search Movies.** Staff click the Movies tab to view the catalog. Typing a keyword in the search box and clicking Search runs a parameterized SELECT that returns matching titles. Output: a refreshed table view.

• **Register a Customer.** On the Customers tab, clicking *Add Customer* opens a form. Filling it in and clicking OK runs an INSERT and refreshes the table. Output: the new row appears at the bottom of the customer list.

• **Process a Rental.** On the Rentals tab, *Add Rental* opens a form taking CustomerID, MovieID, rental date, due date, and status. Submitting runs an INSERT into Rentals. Output: rental appears in the list with status "Active".

• **Process a Return.** On the Returns tab, *Add Return* opens a form taking RentalID, return date, and late fee. The corresponding Rental can then be updated to status "Returned" from the Rentals tab. Output: a Returns row plus a status update.

• **Submit a Rating.** On the Ratings tab, *Add Rating* takes CustomerID, MovieID, score (1-5), and review date. Output: rating recorded; the average score for a movie can be displayed via the *Show Average* button.

• **Edit / Delete Records.** Selecting a row and clicking Edit opens a pre-filled form. Submitting runs an UPDATE. Delete confirms with a dialog and then runs a DELETE.

6. Non-Functional Requirements

• **Performance.** Indexes are automatically created on primary keys and the UNIQUE Email column. Filter operations on the Rentals and Ratings tables run against indexed columns where appropriate.

• **Security.** Database credentials are kept in a single class (`DatabaseConnection`) for easy rotation. All queries use PreparedStatements with parameter binding, eliminating SQL injection through user input.

• **Maintainability.** The three-tier separation localizes change. Adding a new field to Movies, for example, requires editing only the Movie model, MovieDAO, and the Movies form — the rest of the application is untouched.

• **Usability.** Common actions are reachable in two clicks. All forms validate basic input (numeric fields, date format) before sending the SQL statement, and confirmation dialogs guard destructive actions.

7. Tools and Technologies

• **Database:** MySQL Server 8.0

• **Database API:** JDBC with the MySQL Connector/J driver (`com.mysql.cj.jdbc.Driver`)

• **Programming Language:** Java (JDK 11 or later)

• **UI Toolkit:** Java Swing (JFrame, JTabbedPane, JTable, JOptionPane, JDialog)

• **Development Environment:** IntelliJ IDEA / Eclipse / VS Code

• **Version Control:** Git

8. Conclusion

The Movie Rental Database System ties together every major theme covered in CS 157A. The schema is derived from an entity-relationship diagram by translating entity sets to tables and relationships to foreign-key links. Each relation is verified to be in BCNF. The Java application sits on top of this schema and exposes the four core SQL operations through a JDBC-driven application layer, demonstrating the three-tier architecture and the principle of data independence in practice.

The result is a functional and well-structured starting point that could be extended in several directions: swapping Swing for a JSP/Servlet web front end (touching only the presentation layer), adding triggers or stored procedures for late-fee calculation, or introducing role-based access control on top of the existing tables. Each of these extensions could be made without altering the underlying relational schema, which is the point of building the system this way in the first place.